What Makes a Great AI-Agent Skill?

2026-07-27 • 24 Sources

24	6	0	8
VERIFIED SOURCES	MAJOR FINDINGS	UNSUPPORTED FACTUAL CLAIMS	RUBRIC DIMENSIONS

EXECUTIVE SUMMARY

A great AI-agent skill is a compact, testable operating contract for one recognizable user goal. It is not simply a large prompt, a documentation dump, or a collection of tips. Its metadata makes it activate for the right requests and stay inactive for unrelated ones; its body tells the agent what inputs to inspect, which decisions to make, which actions to take, what not to infer, and what finished work looks like. OpenAI, Anthropic, and the open Agent Skills specification converge on this focused, progressively disclosed design [1], [3], [4].

The highest-leverage design rule is **minimum sufficient intervention**. Put only routing and always-needed procedure in `SKILL.md`; move detailed policies, examples, schemas, and templates into directly linked resources; add scripts only when deterministic computation, transformation, or validation materially improves reliability [1], [3], [8], [9]. A good skill constrains the brittle parts of a workflow while leaving the model discretion where judgment is useful. It supplies a preferred path and explicit escape conditions instead of an exhaustive decision tree.

Quality is demonstrated through behavior, not prose. A production-ready evaluation suite tests positive triggers, paraphrased triggers, negative triggers, incomplete inputs, task outcomes, error recovery, and coexistence with other skills. Because agent behavior varies across runs, representative cases need repeated trials and final-state grading rather than one successful transcript [5], [11], [13], [19]. The skill should beat a no-skill baseline on the failures it was created to address.

Security and maintenance are intrinsic quality dimensions. Skills can invoke code and tools, consume adversarial external content, and transmit data. A great skill declares dependencies, treats external text and tool metadata as untrusted, preserves user control for consequential actions, narrows permissions, records validation steps, and is versioned with a rollback path [5], [6], [14], [15], [20], [21].

> **Primary recommendation:** build skills evaluation-first: define one recurring failure and its acceptance tests, write the smallest skill that fixes it, then promote only after trigger, outcome, robustness, security, and coexistence gates pass.
>
> **Confidence:** High for the design principles, because the findings are consistent across current OpenAI and Anthropic documentation, the open specification, independent agent benchmarks, and government security research. Direct comparative studies of alternative `SKILL.md` designs remain limited.

INTRODUCTION

Research question and scope

This report asks: **what makes a great skill for an AI agent such as Codex?** Here, a skill means a reusable filesystem-based package whose `SKILL.md` contains metadata and procedural guidance, with optional references, assets, scripts, and tool dependencies. The scope includes discovery, instruction architecture, tool use, evaluation, security, maintainability, and distribution. It excludes general human skill acquisition, model fine-tuning, and product-specific prompt tricks that do not transfer to reusable agent workflows.

The analysis assumes a technical audience that may create personal, repository-scoped, or distributable skills. It treats OpenAI's current Codex guidance as authoritative for Codex behavior, the Agent Skills specification as the interoperability baseline, and Anthropic's documentation as a mature independent implementation of the same open format [1], [2], [3], [4]. Academic agent research and security benchmarks are used to test whether vendor authoring guidance is consistent with observed model limitations [9], [12], [13], [17], [20].

Methodology

Research used a standard-mode evidence loop: scope, retrieval, source registration, evidence capture, triangulation, outline refinement, synthesis, and claim verification. Twenty-four sources were registered. They comprise current official documentation from OpenAI, Anthropic, Agent Skills, and MCP; first-party engineering reports; peer-reviewed or archival research; and NIST security analysis. Every source has a stable identifier in `sources.jsonl`, and a short supporting passage or data point is preserved in `evidence.jsonl`.

Major conclusions required support from at least three independently published sources or source clusters. Vendor claims about their own runtime behavior were not generalized without corroboration. Where evidence supports an engineering recommendation rather than a universal causal law, the report labels it as synthesis.

The evidence changed the initial outline by elevating activation quality and coexistence to first-class concerns: enterprise guidance showed that a well-executed workflow can still be a bad skill if it steals unrelated triggers or degrades other installed skills [5].

Key assumptions

The central assumption is that a skill's purpose is to improve repeatable task performance, not to maximize the amount of knowledge available to the model. A second assumption is that current agent hosts use progressive disclosure: metadata is visible for discovery, the main instructions load on activation, and supporting files load only as needed [2], [3]. A third is that the deployed model, tools, permissions, and surrounding skills are part of the effective system; therefore, quality cannot be established by reading `SKILL.md` in isolation. Finally, the report assumes that consequential actions remain subject to the host's user-consent and sandbox policies rather than being authorized merely because a skill requests them [14], [21].

MAIN ANALYSIS

Finding 1: Start with a narrow task and an activation contract

The first mark of quality is fit. A skill should exist because a recurring, recognizable task improves when the agent receives specialized procedure, domain context, or deterministic helpers. OpenAI recommends keeping each skill focused on a recognizable user goal and splitting workflows when their triggers, inputs, or success criteria differ [1]. Anthropic's evaluation-first guidance similarly begins by observing representative tasks without a skill and documenting concrete capability gaps [7]. This leads to a practical threshold: if the intended behavior is a universal repository convention, it belongs in `AGENTS.md`; if it is a one-off constraint, it belongs in the prompt; if it needs live external capabilities, it may need MCP; a skill is the right surface when the value lies in a reusable workflow [2].

This distinction prevents two common failures. The first is the **miscellaneous handbook**: a skill named for a broad profession or file type that contains many loosely connected procedures. It is difficult to trigger accurately, expensive to load, and ambiguous about which branch governs a given request. The second is the **micro-skill swarm**: dozens of near-duplicate skills whose descriptions compete for the same requests. Enterprise guidance warns that a new skill can degrade the installed set by over-triggering or stealing matches from another skill [5]. The correct unit is therefore neither "everything about a domain" nor "one instruction per skill"; it is one user goal with a coherent input-output contract.

The activation contract lives primarily in `name` and `description`. In Codex, metadata is available before the full skill body, and implicit activation depends on the task matching the description [2]. The open specification requires the description to state both what the skill does and when it should be used [3].

OpenAI adds that the body should hold detailed procedure, format, and safety rules, while the description should state the workflow and trigger conditions [1]. These mechanics make the description part of runtime behavior rather than marketing copy.

A strong description has four properties. It names the outcome in user language, includes likely trigger terms, states the relevant artifact or environment, and draws a meaningful boundary. For example, "Create, edit, render, and visually verify `.docx` files; use for Word document requests where layout matters" is more operational than "Helps with documents." Negative scope is useful when nearby intents are easy to confuse: a live-Excel control skill should say that it is not for standalone `.xlsx` generation. However, exhaustive "do not use" lists can themselves dilute the key trigger. The highest-value use case and nouns should come first because hosts may shorten descriptions when many skills are installed [2].

Trigger quality is bidirectional. Under-specified descriptions miss relevant requests; over-broad descriptions activate on unrelated ones [19]. Evaluation must therefore measure both recall and precision. A minimum trigger set includes direct requests, natural paraphrases, requests that mention the artifact but ask only for advice, adjacent workflows that belong to another skill, and ambiguous cases that require clarification. For a robust estimate, each case should run multiple times because model outputs and routing decisions vary [11], [19]. The target is not universal activation; it is high activation on intended work and reliable restraint elsewhere.

The resulting design principle is **one job, many realistic phrasings**. A great skill is easy for a user to name, easy for a host to recognize, and easy for a maintainer to test. Its scope can be summarized without reading the body. If authors cannot write a precise activation description, the workflow boundary is probably not yet coherent enough to implement.

Finding 2: Progressive disclosure is the core information architecture

Skills are valuable partly because they can carry more knowledge than should be pasted into every prompt. That advantage disappears if the skill loads everything at once. The Agent Skills specification defines a three-level architecture: lightweight metadata at startup, the full `SKILL.md` after activation, and supporting resources only when required [3]. Codex follows the same pattern [2]. Anthropic describes context as a finite resource and advises finding the smallest high-signal set of tokens that produces the desired behavior [8].

This architecture is not only a cost optimization. Long-context research shows that more available text does not guarantee better use of that text. Liu et al. found that performance often fell when relevant information was placed in the middle of long contexts, including for explicitly long-context models [9]. In their multi-document task, adding retrieved documents produced diminishing returns even while retrieval recall continued to rise [9]. The direct implication for skill authors is conservative: do not assume that a model will reliably discover one critical rule buried in a long manual. Put always-applicable constraints close to the top-level workflow and route conditionally to focused references.

`SKILL.md` should therefore act as an index and control plane. It needs the activation-independent invariants, the main workflow, decision points, stop conditions, required validation, and direct links to optional material. Detailed API schemas, policy tables, format specifications, domain encyclopedias, and extended examples belong in `references/`. Templates and files to copy or transform belong in `assets/`. Deterministic computation and file-processing helpers belong in `scripts/` [1] , [3] , [4] . The main file should tell the agent exactly when to read or run each resource.

Good routing is shallow. The open specification recommends direct relative references and warns against deeply nested chains [3] . Anthropic's authoring guide explains that agents may preview rather than fully read files discovered through nested references, which can leave the real instruction unseen [4] . Every essential reference should be reachable directly from `SKILL.md`, and long reference files should begin with a table of contents. File names should describe their role-`invoice_schema.md` and `validate_invoice.py` are better than `notes2.md` and `helper.py`.

Progressive disclosure also supports variant control. Suppose a document skill can create, edit, redline, and visually review files. The top-level workflow can first classify the operation, then route creation to one guide and redlining to another. The model loads only the branch relevant to the task. This reduces conflicting instructions and makes each branch independently testable. Conditional routing is preferable to presenting five equivalent libraries or techniques and asking the model to choose from scratch. A documented default with an escape condition reduces variance without eliminating useful judgment [4] .

The quality test is whether every loaded token earns its place. Information belongs in `SKILL.md` when omitting it would predictably harm most invocations. It belongs in a reference when it is necessary only for a class of tasks. It belongs outside the skill when the base model already knows it, it changes too frequently to maintain safely, or it can be obtained from an authoritative live tool. This produces a skill that is concise at the point of action yet deep when the task actually needs depth.

Finding 3: Specify the brittle decisions, not every thought

A useful skill turns intent into an executable workflow. OpenAI's authoring guidance says the instructions should identify expected inputs, required steps, the user-visible output, facts the model must not infer, conditions for asking, stopping, or declining, and supporting files to consult [1] . Tool-integrated workflows also need unambiguous input and output contracts; Anthropic recommends enforcing those contracts with strict data models where possible [23] . These elements define observable behavior without prescribing hidden reasoning.

The central design choice is the **degree of freedom**. Too little structure leaves the agent to rediscover a fragile procedure every time. Too much structure creates a brittle flowchart that cannot adapt when files, tools, or user intent differ from the author's examples. Strong skills are rigid around safety, irreversible state changes, data contracts, and acceptance criteria; moderately prescriptive around tool sequence and preferred libraries; and flexible around search tactics, code organization, or prose where multiple answers can be valid. This autonomy gradient is more useful than a blanket instruction to "follow these steps exactly."

Imperative, outcome-linked steps are easier to execute than abstract principles. "Render every page to PNG and inspect it before delivery" defines an action and gate. "Ensure high quality" does not. "If the workbook is an active Excel session, use live-control; otherwise create a standalone file" defines a branch. "Use the best spreadsheet approach" pushes the routing problem back to the model. A procedural skill should expose the few decisions that materially change the workflow and state the evidence required at each gate.

The workflow should include feedback loops where defects are visible only after execution. Code should be tested, documents rendered, generated data parsed, citations verified, and deployment state inspected. Research on ReAct supports the general pattern of interleaving action with observations so plans can be updated and exceptions handled [24]. AgentBench likewise identifies long-horizon reasoning, decision-making, and instruction following as obstacles to usable agents [12]. A skill can compensate by breaking work into milestones that generate concrete observations instead of relying on a long unverified plan.

Examples are most valuable when format or judgment is hard to state compactly. A small set of representative input-output pairs can establish tone, granularity, naming conventions, or error handling more efficiently than pages of adjectives [4]. Examples should cover the normal path and one or two discriminating edges, not merely repeat the instructions. They should use realistic data while avoiding secrets, unstable endpoints, or organization-specific facts that will silently expire.

Stop conditions are equally important. A great skill says when missing information is genuinely blocking, when a safe default is acceptable, when a retry is justified, and when user approval is required. This prevents two opposite failure modes: unnecessary questioning that stalls routine work and unauthorized assumptions that alter scope or external state. For consequential workflows, the skill should preserve a clear handoff to the user; OpenAI's agent guide treats human intervention as a critical safeguard, particularly while failures and edge cases are still being discovered [18].

The best instruction body thus resembles a good runbook: short enough to scan, precise at failure-prone boundaries, explicit about artifacts and gates, and adaptable between them. Its purpose is not to narrate everything a capable model already knows. It is to transfer the local procedure, risk judgment, and definition of done that the model would otherwise lack.

Finding 4: Use scripts and tools only where they buy determinism

Optional code can turn a skill from advice into a reliable capability, but code is not automatically an upgrade. OpenAI recommends instructions by default and scripts when deterministic computation or file processing is needed; it explicitly warns against adding a script when existing tools and instructions already solve the task reliably [1]. Anthropic similarly recommends pre-made utilities because they can save context and time while improving repeatability, but requires explicit dependencies, error handling, and clear execution intent [4].

The strongest candidates for scripts are operations with a narrow contract and an objectively checkable result: parsing a fixed format, validating a schema, rendering an artifact, transforming files, computing a checksum, or running a standardized test. These are places where generated one-off code adds variance

without adding useful judgment. A script should accept explicit inputs, produce a documented output, return meaningful exit codes, avoid hidden global state, and provide errors that tell the agent what can be corrected. If it modifies data, it should support a dry run or operate on a recoverable copy when practical.

The skill must distinguish **execute** from **read**. If a helper is an implementation detail, instruct the agent to run it with the exact command and arguments. If the source code contains logic the agent must understand or adapt, label it as a reference. Ambiguity can lead the model to load a large program into context instead of using it, or to execute code whose behavior should have been inspected first. The same principle applies to MCP tools: declaring a dependency makes a capability available, but the skill still needs to say which tool to use, in what sequence, and how to treat missing or ambiguous results [1].

Tool descriptions and schemas form part of the agent-computer interface. Anthropic's broader agent guidance emphasizes simple designs, transparent workflows, and carefully documented, tested tools [10]. Its tool-authoring guidance recommends clear input and output definitions with strict schemas [23]. A skill should therefore avoid vague tool references such as "look up the customer." It should name the authoritative tool, required identifiers, expected response fields, pagination behavior, and fallback when the tool returns no match. Live data and authorization belong in the server; reusable sequencing and output rules belong in the skill [1].

Scripts also create a maintenance and security burden. Dependencies can disappear, platform assumptions can fail, and code can access more of the environment than the prose suggests. The skill should list prerequisites, prefer bundled or workspace-provided runtimes, use portable paths, pin critical behavior where reproducibility matters, and include a small smoke test. Reviewers need to inspect all bundled files, not just `SKILL.md`, because a harmless description can conceal broad filesystem or network behavior [5], [6].

The decision rule is simple: add code when it converts a probabilistic subtask into a stable interface with a measurable benefit. Keep judgment in the model and mechanics in deterministic helpers. If the script cannot be tested independently, cannot explain failure, or merely regenerates code the model could already write safely, it is increasing the skill's attack surface and maintenance cost without establishing quality.

Finding 5: Evaluation-not author confidence-establishes quality

Skill development should begin with a failure that can be reproduced. Anthropic's skill guidance recommends running representative tasks without the skill, recording capability gaps, building evaluations, establishing a baseline, and then writing the minimum instructions needed to improve results [4], [7]. This reverses the common sequence of writing an encyclopedic manual and testing it afterward. It also creates a falsifiable reason for every instruction: removing or changing it should affect an observed behavior or quality metric.

A complete suite tests five layers. **Activation tests** measure whether direct and paraphrased intents trigger the skill and unrelated intents do not. **Procedure tests** inspect whether the agent consults required files, uses the intended tools, respects stop conditions, and performs validation. **Outcome tests** grade the final artifact or environment state. **Robustness tests** vary input size, missing data, tool errors, and realistic edge cases.

Coexistence tests run the skill alongside the installed set to detect trigger theft, contradictory instructions, or performance regressions [5]. A skill that passes only when invoked explicitly has not demonstrated implicit discoverability; a skill that activates correctly but produces a broken artifact has not demonstrated task quality.

Final-state grading matters because transcripts can be persuasive while outcomes are wrong. Anthropic's evaluation framework distinguishes the trace from the environment's final state: an agent can claim that a reservation exists even when no database record was created [11]. For coding, unit tests and repository diffs are natural graders. For documents, render-and-inspect checks catch layout defects that text extraction misses. For research, groundedness, source quality, coverage, and citation integrity require separate graders. Human or model grading can assess open-ended quality, but deterministic checks should cover every property that can be measured directly.

Repeated trials are necessary. Agent outputs vary, so one green run overstates reliability [11]. The Agent Skills description guide suggests multiple runs when measuring trigger behavior [19]. The `tau-bench` benchmark formalizes consistency with pass-at-k-style evaluation and reported that then-current function-calling agents succeeded on fewer than half of tasks, with much lower reliability across repeated trials in one domain [13]. Although benchmark results age quickly, the methodological point remains: report distributions or pass rates, not a curated success.

Representative cases matter more than raw test count. AgentBench spans eight interactive environments because multi-turn reasoning and action differ from static text generation [12]. For a narrow skill, a smaller matrix can be sufficient if it covers distinct risks. A practical starting suite contains three to five positive triggers, three negative or adjacent triggers, two incomplete inputs, three core task cases, two edge or failure cases, and one coexistence run. High-risk skills add adversarial inputs, permission failures, and rollback checks. Each case needs explicit acceptance criteria that a second reviewer could apply consistently.

Metrics should map to failure modes:

DIMENSION	SUGGESTED MEASURE	TYPICAL FAILURE
Trigger precision	correct non-activation / negative cases	skill steals unrelated work
Trigger recall	correct activation / intended cases	skill is never discovered
Procedure adherence	required gates completed	validation or approval skipped
Outcome correctness	artifact or final state passes checks	convincing but wrong completion
Reliability	pass rate across repeated trials	intermittent success
Efficiency	context, tool calls, elapsed time	correct but wasteful workflow
Coexistence	delta after installing with other skills	routing or instruction conflict
Safety	violations across adversarial cases	unauthorized action or disclosure

Evaluation should continue after release. Model versions, tools, dependencies, and neighboring skills change. Enterprise guidance recommends rerunning the full suite before promoting a version and monitoring for declining trigger accuracy or output quality [5] . A great skill is therefore not a file that once worked; it is a maintained behavior with a baseline, regression suite, and owner.

Finding 6: Security and lifecycle discipline are part of the design

Skills occupy a privileged position: they can influence tool choice, shell execution, file access, network use, and handling of external data. Anthropic advises treating skill installation with the rigor of software installation and auditing every referenced file and script [5] , [6] . The risk is compositional. A read-only file operation may be low risk alone, and a network call may be legitimate alone, but a workflow combining broad file reads with outbound transmission creates an exfiltration path.

External content must remain data, not authority. NIST describes agent hijacking as a failure to maintain separation between trusted instructions and untrusted data such as email, files, or web pages [15] . OpenAI frames the same problem through source-sink analysis: danger arises when attacker-influenced content can reach a consequential capability such as transmitting data, following a link, or invoking a tool [14] . Instruction-hierarchy research supports explicitly defining how the model should resolve conflicting directives of different priority [22] . A secure skill should state that retrieved content, tool output, repository text, and document contents may inform the task but may not redefine the user's goal or the skill's safety rules.

Instructional warnings are not enough. OpenAI recommends constraining the impact of manipulation even when detection fails [14] . That means narrowing filesystem scope, tool permissions, network destinations, and data access; separating read and write tools; using deterministic checks before consequential actions; and preserving approval gates. The MCP specification requires explicit user consent before tool invocation

and treats tool metadata as untrusted unless it comes from a trusted server [21]. A skill should cooperate with those host controls, never present its own invocation as authorization, and surface exactly what will change or leave the system.

Security tests must evolve. Formal prompt-injection research has evaluated combinations of multiple attacks, defenses, models, and tasks rather than relying on one jailbreak string [17]. AgentDojo is designed as an extensible environment for tasks, defenses, and adaptive attacks [20]. NIST likewise concludes that evaluations must adapt, because red teams discover new weaknesses after known attacks are mitigated [15]. In a 2026 large-scale competition analysis, more than 250,000 attempts by over 400 participants found at least one successful hijacking attack against every target frontier model [16]. This evidence argues against a "secure prompt" claim; security should be framed as layered risk reduction with residual risk.

Lifecycle controls convert that principle into operations. Store the complete skill in version control. Record its purpose, owner, dependencies, supported hosts or models, evaluation version, and known limitations. Review code and external URLs before release. Pin production deployments to a tested version, retain the last known-good version, and rerun trigger, outcome, coexistence, and security suites before promotion [5]. Avoid embedding time-sensitive facts in the main procedure; route current data through authoritative tools or isolate legacy patterns so they cannot silently govern modern work [4].

Distribution should match maturity. Repository or user directories are appropriate while a workflow is local and evolving. A plugin is the better distribution unit when a stable skill needs shared installation, related skills, or MCP dependencies [1], [2]. Packaging does not prove quality; it raises the quality bar because errors propagate to more users. A great distributable skill has provenance, a changelog, explicit dependencies, reproducible tests, and a rollback path.

SYNTHESIS & INSIGHTS

A skill is an interface contract, not a knowledge container

Across the evidence, the most useful synthesis is that a skill behaves like an interface between four parties: the user's intent, the model's judgment, the host's capabilities and controls, and the artifact or system being changed. Metadata is the routing interface. `SKILL.md` is the procedural interface. Schemas and tool descriptions are the machine interface. Acceptance tests are the outcome interface. Most weak skills leave one of these implicit.

This framing explains why writing quality alone is insufficient. A beautifully written body cannot compensate for a description that activates on the wrong tasks. A correct workflow cannot compensate for a missing dependency. A successful transcript cannot compensate for a broken final state. A secure host cannot compensate for a script that reads unrelated secrets and sends them to an arbitrary endpoint. Greatness is the product of the interfaces working together, not the maximum score on any single dimension.

The governing principle is minimum sufficient intervention

The evidence supports a "minimum sufficient intervention" rule. Begin with the base model and available tools. Observe a recurring failure. Add the smallest instruction, reference, example, or deterministic helper that makes the failure reliably disappear. Retain that addition only if the evaluation improvement exceeds its context, complexity, and maintenance cost. This is consistent with evaluation-first skill development, context economy, and the broader advice to add agentic complexity only when it demonstrably improves outcomes [7] , [8] , [10] .

This principle yields an escalation ladder:

1. Clarify the activation description.
2. Add one explicit invariant or decision boundary.
3. Add a concrete example for a hard-to-describe output.
4. Add a focused reference for conditional depth.
5. Add a validation or feedback step.
6. Add a script for deterministic mechanics.
7. Add MCP only for live data, authorization, or controlled external actions.
8. Package as a plugin only when distribution and dependency management justify it.

Each rung should solve an observed problem. Skipping directly to scripts, connectors, or large reference libraries often creates a more impressive package without producing a more reliable skill.

Great skills compress organizational experience

The durable value of a skill is not generic domain knowledge. Models already possess broad knowledge, and current facts can often be retrieved. The scarce information is the organization's procedure: which source is authoritative, which ambiguity changes the workflow, which validation caught past failures, which action requires approval, and what a reviewer considers complete. A skill is excellent when it compresses that experience into a workflow that another agent instance can execute and verify.

This also suggests a maintenance strategy. Do not append every correction. Classify feedback. Repeated routing errors update the description. Repeated procedural errors update the main workflow. Rare domain details become references. Mechanical failures become scripts or validators. Universal repository conventions move to `AGENTS.md`. Expired facts are deleted or fetched live. This keeps the skill small while its reliability improves.

A practical quality rubric

The following 100-point rubric translates the findings into a release gate:

DIMENSION	WEIGHT	FULL-CREDIT STANDARD
Task fit and scope	15	one recognizable goal; correct surface; explicit non-goals
Trigger quality	15	precise description; positive, paraphrase, negative, and ambiguous cases pass
Context architecture	10	concise main file; direct routing; no buried critical rules
Workflow clarity	15	inputs, outputs, decisions, stop conditions, and definition of done are explicit
Execution reliability	10	deterministic helpers where justified; dependencies and errors handled
Verification	15	final-state checks, repeated trials, baseline comparison, regression suite
Safety and user control	10	trust boundaries, least privilege, approvals, and adversarial tests
Maintainability	10	owner, versioning, provenance, coexistence tests, rollback, current references

A score is useful only if backed by evidence. A skill should not receive more than half credit in verification without recorded runs, or more than half credit in trigger quality without negative cases. Any unresolved critical security violation is a release blocker regardless of total score. For production use, a reasonable promotion threshold is 80/100 with no zero-score dimension; high-risk workflows should require independent review and explicit safety gates.

LIMITATIONS & CAVEATS

Counterevidence Register

The source base is strong on convergent guidance but weaker on controlled comparisons of alternative skill designs. OpenAI and Anthropic document how their systems are intended to work and report lessons from internal use, but public experiments that isolate one `SKILL.md` variable at a time are scarce. The 100-point rubric is therefore a synthesis for engineering use, not a validated psychometric scale.

Model and host behavior changes. Trigger routing, context limits, skill metadata, permissions, and plugin packaging may evolve after this report. Some guidance is implementation-specific: for example, Codex's discovery locations and metadata budgets should not be assumed to apply to every Agent Skills host [2].

The open specification establishes a portable minimum, while optional metadata and tool policies may have different support across clients [3].

Long-context findings provide a reason to prefer focused information architecture, but the cited experiments include older models and tasks that are not identical to skill execution [9]. They do not prove that every longer skill performs worse. The defensible conclusion is narrower: additional context has an opportunity cost, critical rules should not be buried, and skill length should be tested rather than presumed harmless.

Agent benchmarks also age quickly. The specific success rates in `tau-bench` describe the evaluated systems and time, not current frontier performance [13]. Their lasting contribution is methodological: evaluate dynamic tool use, domain rules, final state, and consistency across trials. Likewise, security research demonstrates residual vulnerability and the need for layered controls; it does not establish that every skill consuming external content will be compromised [14], [16], [20].

No sources supported a universal optimal line count, number of examples, or evaluation-suite size across all skills. The specification's recommended limits and authoring checklists are sensible defaults, not substitutes for measured behavior. High-risk or unusually broad workflows may need more tests, independent security review, and stricter permission boundaries than this general rubric describes.

RECOMMENDATIONS

Build evaluation-first

Write three representative task cases before extensive instructions: one normal case, one difficult edge, and one case that currently fails. Record the no-skill baseline. Add the minimum procedure necessary to pass, then expand only when a new observed failure justifies it [4], [7], [11]. Define success in terms of the final artifact or environment state, not whether the transcript sounds correct.

Treat the description as a classifier

Draft the description from actual user phrasing. State the outcome, artifact or environment, and trigger condition in the first sentence. Add a boundary only for a likely adjacent confusion. Test direct requests, paraphrases, negative cases, and ambiguous cases multiple times [1], [5], [19]. If two skills compete for the same cases, narrow or consolidate them before adding more trigger prose.

Design `SKILL.md` as a shallow router

Keep the main file focused on invariants, the primary workflow, decision points, stop conditions, validation, and direct resource links. Move conditional depth into focused references one level away. Put templates in assets and mechanics in scripts [1], [3], [4]. Review every paragraph with a simple question: would most invocations fail or become materially worse without this in immediate context?

Encode an autonomy gradient

Use strict rules for permissions, irreversible changes, data contracts, and acceptance gates. Provide preferred defaults for tools and common paths, with explicit escape conditions. Leave the model discretion for exploratory tactics and qualitative judgment. State what the agent must not infer and when it must ask, stop, retry, or hand control to the user [1], [18].

Make every critical operation verifiable

Pair generation with a checker: run tests for code, render visual artifacts, parse structured outputs, compare final state, or validate citations. Use deterministic scripts when they remove mechanical variance, but document dependencies, inputs, outputs, exit behavior, and error recovery [1], [4], [10], [23]. A helper that cannot be independently tested should not become a hidden foundation of the workflow.

Release through explicit gates

Before promotion, require:

- trigger precision and recall cases;
- task outcome and final-state checks;
- repeated trials for nondeterministic behavior;
- edge, failure-recovery, and coexistence cases;
- review of every file, dependency, command, URL, and requested permission;
- adversarial tests for untrusted external content;
- an owner, version, changelog, known limitations, and rollback path.

Treat a critical safety failure as blocking even when the overall quality score is high [5], [14], [15], [20], [21]. Rerun the suite when the skill, model, tools, dependencies, or neighboring skill set changes.

Use the right distribution surface

Keep an evolving personal workflow in a user skill and a project-specific workflow in the repository. Move universal repository conventions to `AGENTS.md`. Package a stable capability as a plugin when it needs team distribution, multiple related skills, or MCP dependencies [1], [2]. Installation scope should follow the smallest audience that needs the behavior.

METHODOLOGY APPENDIX

Research process

The run began on 2026-07-27 in standard mode. The question was decomposed into discovery, context design, procedural guidance, deterministic execution, evaluation, security, and lifecycle management. Searches prioritized official OpenAI documentation for Codex behavior, official Agent Skills and Anthropic material for cross-implementation design, original academic papers for model and benchmark evidence, and NIST for independent security analysis.

Sources were registered before synthesis using stable identifiers derived from canonical URLs or arXiv IDs. A compact evidence passage or data point from each source was stored in an append-only ledger with a locator and retrieval query. The final source set contains 24 entries spanning five source types: official documentation and standards, first-party engineering, peer-reviewed or archival research, government research, and protocol specifications.

Triangulation and outline refinement

The initial outline emphasized instruction writing, resources, and tests. Triangulation added two major concerns. First, OpenAI and Anthropic guidance showed that the description controls implicit discovery, while enterprise guidance identified over-triggering and coexistence as independent failure modes [1], [5], [19]. Second, security sources showed that skill quality must include trust boundaries, permissions, and adversarial testing rather than treating security as a deployment afterthought [14], [15], [20], [21].

Major findings were retained only when at least three source clusters supported the conclusion or when a source authoritatively documented its own runtime behavior and independent evidence supported the engineering implication. Specific benchmark results were treated as time-bound. Recommendations and the quality rubric are explicitly synthetic: they integrate the evidence but are not quoted standards.

Claims-Evidence Table

ATOMIC CONCLUSION	INDEPENDENT SUPPORT CLUSTERS	STATUS
Descriptions govern discovery and require narrow boundaries	OpenAI runtime guidance [1], [2]; Agent Skills guide [19]; Anthropic enterprise evaluation [5]	Supported
Progressive disclosure improves context architecture	Open specification [3]; Codex documentation [2]; context research [8], [9]	Supported
Workflows need explicit interfaces and feedback	OpenAI workflow guidance [1]; tool-interface guidance [23]; ReAct [24]; AgentBench [12]	Supported
Deterministic helpers are justified by measured reliability	OpenAI authoring guidance [1]; Anthropic authoring guidance [4]; agent design guidance [10]	Supported
Evaluation needs final-state checks and repeated trials	Anthropic eval framework [11]; enterprise guidance [5]; AgentBench and tau-bench [12], [13]	Supported
Security requires trust boundaries and layered controls	OpenAI security [14], [22]; NIST [15], [16]; AgentDojo and MCP [20], [21]	Supported

Verification and limitations

The report's numbered citations map to `sources.jsonl`; evidence passages map to `evidence.jsonl`; atomic conclusions and support links are recorded in `claims.jsonl`. Automated checks validate required sections, bibliography coverage, citation formatting, and claim-support presence. The HTML and PDF are generated from the Markdown source and visually inspected after rendering. Remaining limitations are described in the report's Limitations & Caveats section.

BIBLIOGRAPHY

[1] OpenAI (2026). Build skills.

[2] OpenAI (2026). Customization: Skills.

[3] Agent Skills project (2026). Agent Skills Specification.

[4] Anthropic (2026). Skill authoring best practices.

[5] Anthropic (2026). Skills for enterprise.

[6] Anthropic (2026). Agent Skills overview.

[7] Barry Zhang et al. (2025). Equipping agents for the real world with Agent Skills.

[8] Anthropic (2025). Effective context engineering for AI agents.

[9] Nelson F. Liu et al. (2024). Lost in the Middle: How Language Models Use Long Contexts.

[10] Erik Schluntz & Barry Zhang (2024). Building Effective AI Agents.

[11] Anthropic (2026). Demystifying evals for AI agents.

[12] Xiao Liu et al. (2023). AgentBench: Evaluating LLMs as Agents.

[13] Shunyu Yao et al. (2024). tau-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains.

[14] Thomas Shadwell & Adrian Spanu (2026). Designing AI agents to resist prompt injection.

[15] NIST (2025). Strengthening AI Agent Hijacking Evaluations.

[16] NIST CAISI (2026). Insights into AI Agent Security from a Large-Scale Red-Teaming Competition.

[17] Yupei Liu et al. (2023). Formalizing and Benchmarking Prompt Injection Attacks and Defenses.

[18] OpenAI (2025). A practical guide to building agents.

[19] Agent Skills project (2026). Optimizing skill descriptions.

[20] Edoardo Debenedetti et al. (2024). AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents.

[21] Model Context Protocol (2025). Model Context Protocol Specification: Security and Trust and Safety.

[22] Eric Wallace et al. (2024). The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions.

[23] Ken Aizawa (2025). Writing effective tools for agents - with agents.

[24] Shunyu Yao et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models.